

Milestone Delivery #5
CSE-2102 Group Project - Spring 2025
API Stubs

Deadline: 11:59pm, April 3

Git

- Use your existing team UConn GitHub repo for this milestone and for code delivered in future milestones.
- Create two folders in the root of your repo (if you have not already done so): frontend, backend
- Place the API code into the "backend" folder.
- (NOTE: you are **NOT** creating a new git repo! You are creating folders within your existing repo!!!)
- Use Pull Requests for all merges to "develop" and "main". Assign the Pull Request to someone else on your team. That other team member will do a code review on your code prior to accepting the pull request.

API

- All previous milestones have been building toward the implementation. The API you create should be what you designed. If you realize you missed something in the design, that's okay. Go back and modify the appropriate design artifact in the "docs" folder to reflect this change.
- Implement all the needed API methods in your design using Python Flask. Organize your API code in a way that is clean and succinct. For instance, don't have one giant python file for everything. For instance, you might organize your API code as follows:
 - main.py - Place all your routes into this file
 - Create separate python files for each major grouping of APIs. For instance, you might have a "user.py" for API methods relating to user maintenance and "products.py" for all API methods relating to products. In this example, the routes in main.py would call methods in user.py and products.py.
 - This is just an example
- Create stubs and/or mocks for each API call. That will allow you to test the connectivity or your APIs. You will use these stubs / mocks in your unit tests.
- (Note: you are not connecting to a database for this milestone. That will come in the next milestone.)

GitHub Actions (GHA)

- Create a CICD pipeline to build your API. The pipeline should run on all pushes to "main" or "develop".
- Use pylint to perform static code analysis in your CICD pipeline. Fix any linting errors or warnings in your code. (Do not add "ignore" statements into your pylint configuration.)
- Create pytest unit tests for each API call and automate the testing in your CICD pipeline.

- Your pytest should make http calls to your api. Do not simply call your methods as python function calls in your unit tests.
- Note: in your CICD pipeline make sure you use the "./backend" path for your linting and testing in your pipeline because very soon you will have React code in the "frontend" folder and you don't want pylint and pytest trying run against that code.

Documentation

- Create an API Contract (1 document) and add it to the "docs" folder in git
 - For every API call needed, you want to document the "contract". The contract should list:
 - The http method and relative route or path (e.g. GET /products)
 - The input (and the data type(s)) for the API call
 - The output response (and the data type(s)) of the API call

The "Vibe Coding" & AI Policy

You are encouraged to use AI (vibe coding) for **refactoring, debugging, and fixing** existing code. However:

- **Prohibited:** Using AI to automatically generate entire "fresh" features or core logic from scratch.
- **Requirement:** You must explicitly document how AI was used throughout your project.

Grading

- For grading, the TAs will look at the last successful GitHub Action pipeline run on the "main" branch prior to the due date for this milestone. You will be graded on the code associated with that commit.
 - Be sure to give yourself time to complete the pull request process and for the pipeline to run! In other words, don't try and do this five minutes before midnight.
- Submit a PDF version of your API contract to HuskyCT.
- In the PDF, if AI (vibe coding) was used, specify where and how it was used.
- You will meet with a TA during your lab on the due date. Your team may not yet be complete with all the milestone tasks, but your TA will be looking at:
 - Current status of your project board
 - Current status of your API code
 - Team collaboration and communication.